

DTOs en Java: Buenas prácticas y modernización con records

Juan Cruz Robledo

15 de mayo de 2024



¿Qué es un DTO (Data Transfer Object)?

Un **Data Transfer Object** es un patrón de diseño que define objetos simples para transportar datos entre subsistemas de una aplicación, especialmente a través de límites de red o entre capas.

Los DTOs se centran exclusivamente en la **transferencia de datos**, sin incluir comportamiento de negocio o validaciones complejas.



Propósito Principal

Transportar datos entre capas o procesos sin exponer la lógica interna del dominio, actuando como contenedores simples de información.

Características

Objetos serializables, generalmente inmutables, y desprovistos de comportamiento complejo o lógica de negocio.

¿Por qué usar DTOs?



Separación de Capas

Evita la exposición directa de entidades de dominio, estableciendo límites claros entre capas de la aplicación y mejorando el encapsulamiento.



Seguridad

Previene la filtración de datos sensibles al controlar exactamente qué información se transmite, evitando exponer atributos internos o confidenciales.



Control de Datos

Permite seleccionar precisamente qué datos enviar/recibir, optimizando transferencias y adaptando interfaces a necesidades específicas.



Reducción del Acoplamiento

Minimiza la dependencia entre capas, facilitando cambios en el modelo de dominio sin afectar las interfaces externas.

Los DTOs son especialmente valiosos en arquitecturas distribuidas, APIs REST y aplicaciones multicapa donde la transferencia eficiente y segura de datos es crítica.

Comparación: Entity vs DTO

Usar entidades directamente en controladores o interfaces externas crea problemas significativos de seguridad, rendimiento y mantenibilidad. Es esencial entender las diferencias fundamentales entre estos componentes.



Aspecto	Entity	DTO
Propósito	Representar y persistir datos del dominio	Transferir datos entre capas o sistemas
Responsabilidad	Implementar lógica de dominio y reglas de negocio	Transportar datos sin comportamiento adicional
Relaciones	Puede tener relaciones complejas y bidireccionales	Relaciones simplificadas o referencias por ID
Persistencia	Anotado para JPA/Hibernate y persistencia	Sin anotaciones de persistencia
Exposición	Solo interno, no debe exponerse directamente	Diseñado específicamente para exposición externa

DTO clásico (POJO)

Tradicionalmente, los DTOs se implementaban como **Plain Old Java Objects** (POJOs) con una estructura verbosa que requería escribir manualmente varios elementos repetitivos.

Características del DTO clásico

- Campos privados para encapsulamiento
- Getters y setters para cada atributo
- Constructor(es) con parámetros
- Implementaciones de equals(), hashCode(), toString()
- A veces, implementación de Serializable

```
public class ProductDTO {  
    private Long id;  
    private String name;  
    private BigDecimal price;  
    private String category;  
  
    // Constructor vacío  
    public ProductDTO() {}  
  
    // Constructor con parámetros  
    public ProductDTO(Long id, String name,  
        BigDecimal price, String category) {  
        this.id = id;  
        this.name = name;  
        this.price = price;  
        this.category = category;  
    }  
  
    // Getters y setters (20+ líneas adicionales)  
    public Long getId() { return id; }  
    public void setId(Long id) { this.id = id; }  
  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
  
    // ... y así sucesivamente  
  
    // equals, hashCode, toString  
    // (30+ líneas adicionales)  
}
```

⚠ Este enfoque tradicional genera **boilerplate excesivo**, dificultando la lectura y mantenimiento, especialmente para DTOs con muchos atributos.

Introducción a record (Java 14+)

Desde Java 14, el tipo **record** ofrece una forma concisa de definir clases inmutables de transferencia de datos, eliminando prácticamente todo el boilerplate asociado con los DTOs tradicionales.



Declaración Compacta

Define campos, constructor y métodos de acceso en una sola línea



Inmutabilidad

Todos los campos son final por defecto, garantizando objetos inmutables



Generación Automática

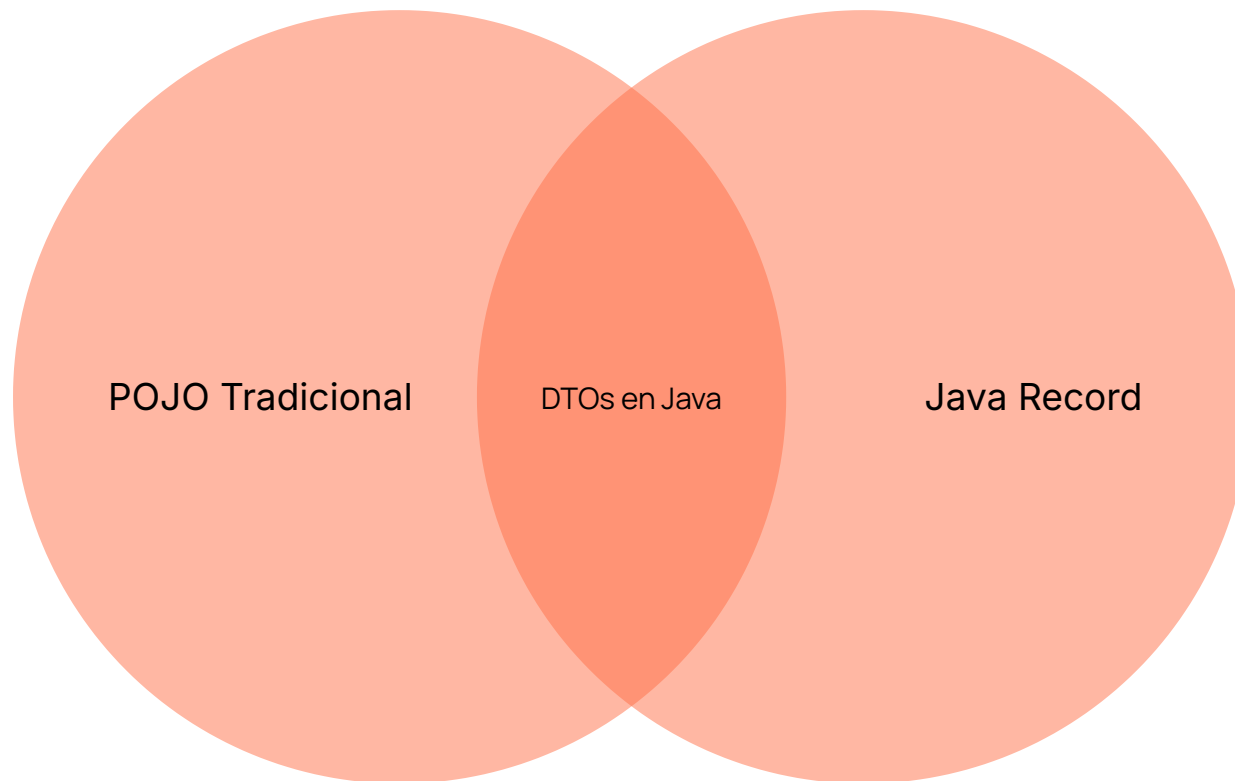
Implementaciones automáticas de equals(), hashCode(), toString() y getters

```
// Definición de un record
public record ProductRecord(
    Long id,
    String name,
    BigDecimal price,
    String category
) {
    // El compilador genera automáticamente:
    // - Constructor canónico
    // - Métodos de acceso (id(), name(), etc.)
    // - equals(), hashCode(), toString()

    // Se pueden agregar métodos adicionales
    public boolean isPremiumProduct() {
        return price.compareTo(
            new BigDecimal("100")
        ) >= 0;
    }

    // También constructores compactos
    public ProductRecord {
        // Validaciones o transformaciones
        if (name == null) {
            name = "Sin nombre";
        }
    }
}
```


Ejemplo comparativo: POJO vs record

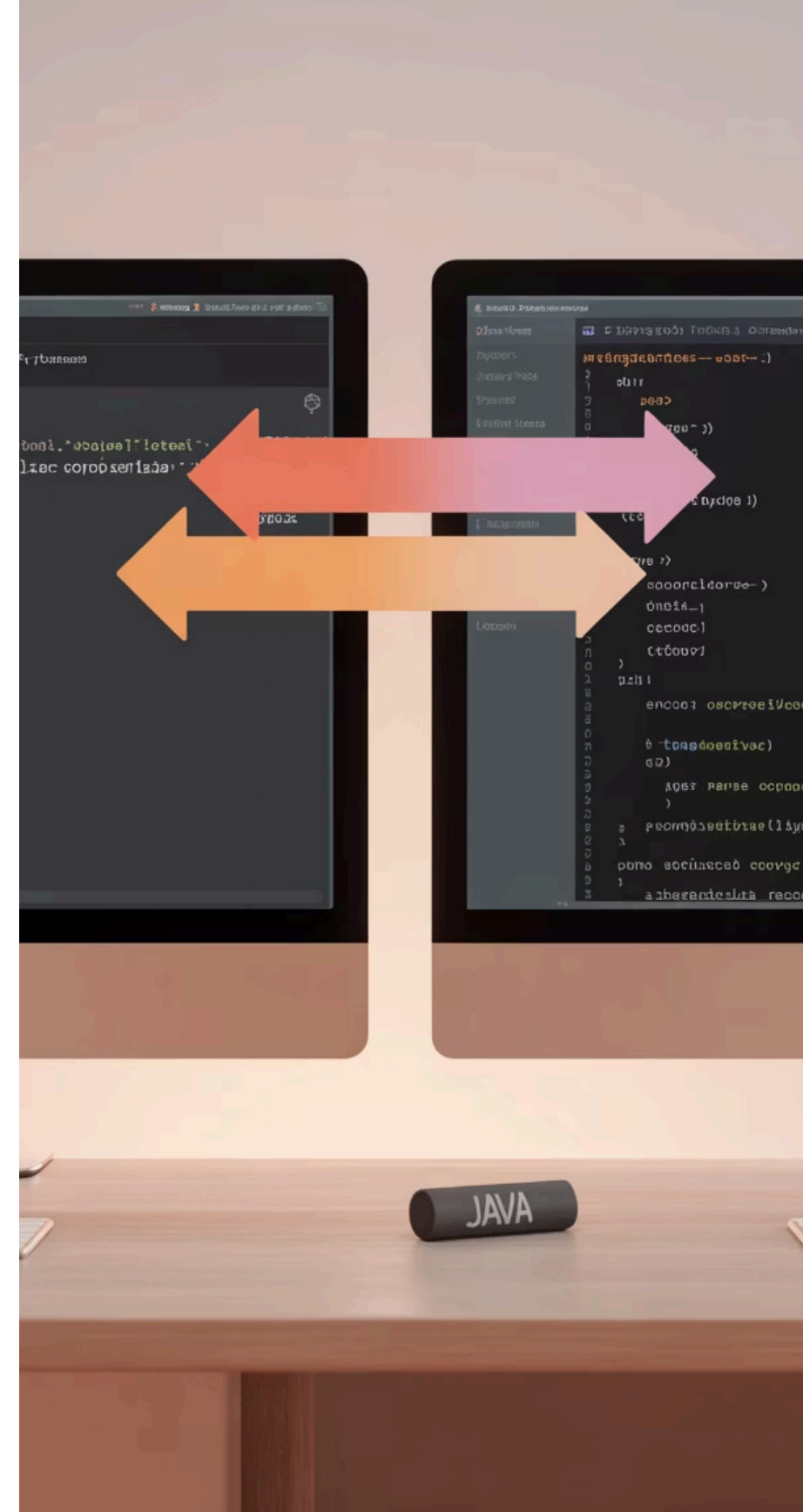


Ventajas del record

- Reducción drástica de líneas de código (~70-80%)
- Mayor legibilidad y mantenibilidad
- Inmutabilidad garantizada por diseño
- Semántica clara: "este objeto es solo para datos"

Características compartidas

- Capacidad para transportar datos
- Compatibilidad con JSON/serialización
- Posibilidad de agregar métodos
- Uso en APIs y comunicación entre capas



Casos en los que record es ideal



DTOs de respuesta en APIs

Los records son perfectos para modelar respuestas de API que no necesitan modificarse una vez creadas, garantizando consistencia en los datos devueltos.



Objetos de valor inmutables

Cuando necesitas objetos cuya identidad está determinada por sus valores y no deben cambiar después de su creación, como configuraciones o parámetros.



Transferencia de datos simples

Para objetos con el único propósito de transportar datos entre capas o componentes sin lógica adicional o comportamiento complejo.



Proyecciones y agregaciones

Ideal para representar resultados de consultas específicas que combinan datos de múltiples entidades o filtran campos específicos.

```
// Ejemplo de DTO para respuesta de API
public record ProductSummaryDTO(
    Long id,
    String name,
    BigDecimal price
){
    // Método estático para convertir desde entidad
    public static ProductSummaryDTO from(Product product) {
        return new ProductSummaryDTO(
            product.getId(),
            product.getName(),
            product.getPrice()
        );
    }
}
```


Limitaciones de record

Aunque los records son muy convenientes para DTOs, tienen limitaciones importantes que debes considerar antes de adoptarlos universalmente.



☐ Sin herencia de clases

Los records no pueden extender otras clases y son implícitamente finales. Solo pueden implementar interfaces, limitando la jerarquía de DTOs.

☐ Campos siempre finales

Todos los campos son inmutables, lo que puede ser inconveniente cuando necesitas modificar propiedades después de la creación.

☐ Compatibilidad con frameworks

Algunos frameworks antiguos o bibliotecas que dependen de JavaBeans pueden no funcionar correctamente con records (JPA, ciertos serializadores).

☐ Personalización limitada

Menor flexibilidad para personalizar la generación de métodos estándar como `equals()`, `hashCode()` o `accessors`.

Validación en DTOs

Los DTOs son el lugar ideal para implementar validaciones que garanticen la integridad de los datos que ingresan a nuestra aplicación, especialmente en APIs públicas.

Java Bean Validation (JSR 380) proporciona un mecanismo estándar para definir validaciones mediante anotaciones, compatible tanto con POJOs tradicionales como con records.



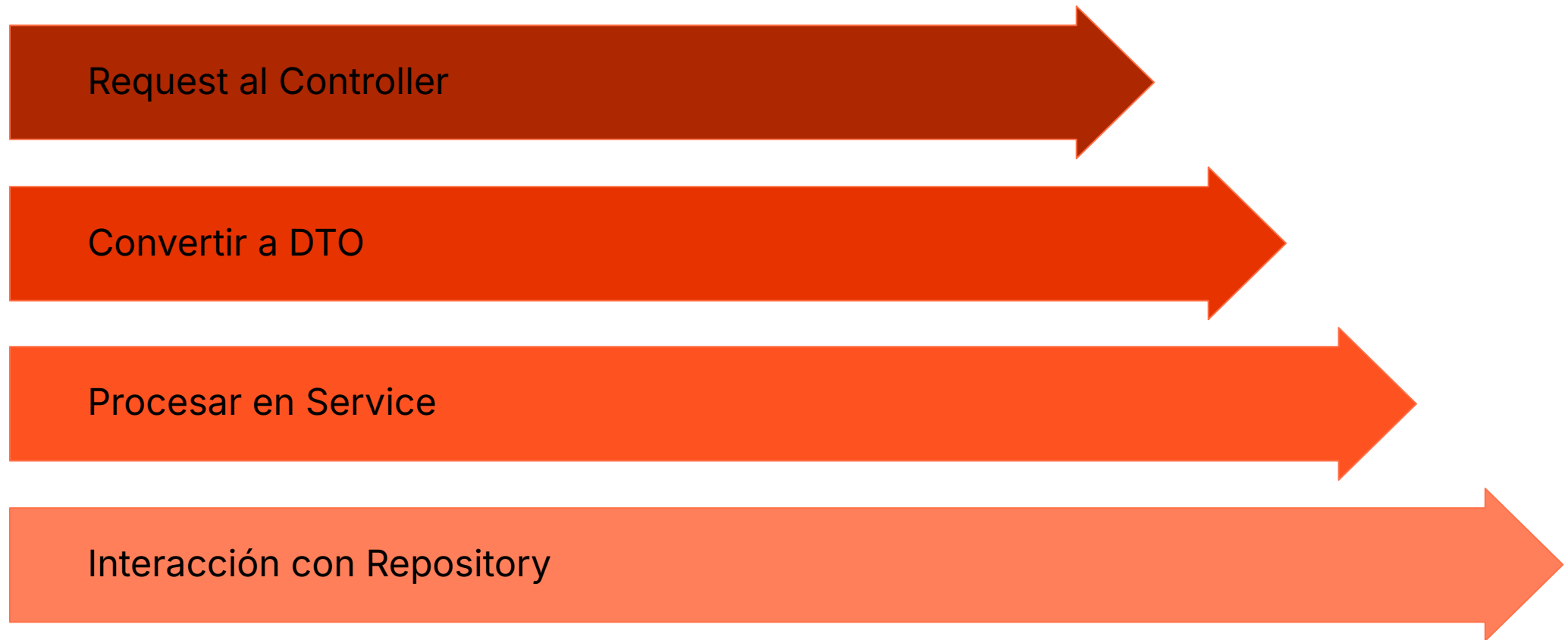
```
// Validación en un record DTO
public record UserRegistrationDTO(
    @NotBlank(message = "El nombre es obligatorio")
    @Size(min = 2, max = 100, message = "El nombre debe tener entre 2 y 100 caracteres")
    String name,

    @NotBlank(message = "El email es obligatorio")
    @Email(message = "Formato de email inválido")
    String email,

    @NotBlank(message = "La contraseña es obligatoria")
    @Size(min = 8, message = "La contraseña debe tener al menos 8 caracteres")
    @Pattern(regex = ".*[A-Z].*", message = "Debe contener al menos una mayúscula")
    @Pattern(regex = ".*[0-9].*", message = "Debe contener al menos un número")
    String password
) {
    // Constructor compacto para validación adicional
    public UserRegistrationDTO {
        if (email != null) {
            email = email.toLowerCase().trim();
        }
    }
}
```

Estructura típica en una API REST

Los DTOs son componentes esenciales en una arquitectura de API REST bien diseñada, actuando como intermediarios entre las capas y protegiendo la integridad del dominio.



Beneficios de esta estructura

- Clara separación de responsabilidades entre capas
- Protección del modelo de dominio
- Facilita cambios independientes en cada capa
- Control sobre los contratos de API

Tipos de DTOs comunes

- **RequestDTO**: Captura datos de entrada
- **ResponseDTO**: Formatea datos de salida
- **PageDTO**: Encapsula resultados paginados
- **ErrorDTO**: Estructura mensajes de error

Mapeo entre Entity y DTO

El mapeo entre entidades de dominio y DTOs es una operación frecuente en aplicaciones bien estructuradas. Existen varias aproximaciones para realizarlo, desde mapeo manual hasta el uso de bibliotecas especializadas.

Mapeo manual

Escribir métodos específicos para convertir entre Entity y DTO. Simple y explícito, pero repetitivo para objetos complejos.

Bibliotecas de mapeo

Herramientas como MapStruct o ModelMapper automatizan la conversión, reduciendo código repetitivo y errores.

```
// Ejemplo de mapeo manual con métodos estáticos
public record ProductDTO(
    Long id,
    String name,
    BigDecimal price,
    String categoryName
){
    // De entidad a DTO
    public static ProductDTO fromEntity(Product product) {
        return new ProductDTO(
            product.getId(),
            product.getName(),
            product.getPrice(),
            product.getCategory().getName()
        );
    }

    // De DTO a entidad (para creación/actualización)
    public Product toEntity(CategoryRepository categoryRepo) {
        Product product = new Product();
        product.setId(this.id);
        product.setName(this.name);
        product.setPrice(this.price);

        // Resolviendo la relación
        Category category = categoryRepo
            .findByName(this.categoryName)
            .orElseThrow();
        product.setCategory(category);

        return product;
    }
}
```

Buenas prácticas con DTOs



Proteger el modelo de dominio

Nunca exponer entidades directamente a través de APIs. Usar siempre DTOs como intermediarios para controlar exactamente qué datos se transfieren y evitar vulnerabilidades de seguridad.



DTOs específicos por caso de uso

Diseñar DTOs distintos para cada operación en lugar de reutilizar un DTO genérico. Esto permite adaptarlos a las necesidades exactas de cada endpoint o función.



Separar DTOs de entrada y salida

Mantener DTOs diferentes para requests y responses, incluso cuando parezcan similares, permite evolucionar independientemente los contratos de entrada y salida.



Versionar DTOs cuando sea necesario

Para APIs públicas, considerar versionado de DTOs para permitir cambios sin romper compatibilidad con clientes existentes.

Recuerda que los DTOs son contratos de comunicación que definen las interfaces de tu aplicación. Diseñarlos cuidadosamente y mantenerlos actualizados es clave para un sistema robusto y flexible.

Conclusiones: DTOs modernos con Java records

Desacoplamiento

Los DTOs son fundamentales para desacoplar capas y servicios, permitiendo que evolucionen independientemente mientras mantienen contratos claros de comunicación.

Simplificación

Los records de Java transforman radicalmente la implementación de DTOs, eliminando boilerplate y mejorando la legibilidad y mantenibilidad del código.

Seguridad

Usar DTOs bien diseñados protege el modelo de dominio, previene la exposición accidental de datos sensibles y fortalece la seguridad de la aplicación.



Al adoptar records para tus DTOs y seguir estas buenas prácticas, lograrás código más conciso, mantenible y seguro, reduciendo significativamente el esfuerzo de desarrollo mientras mejoras la calidad general de tu sistema.